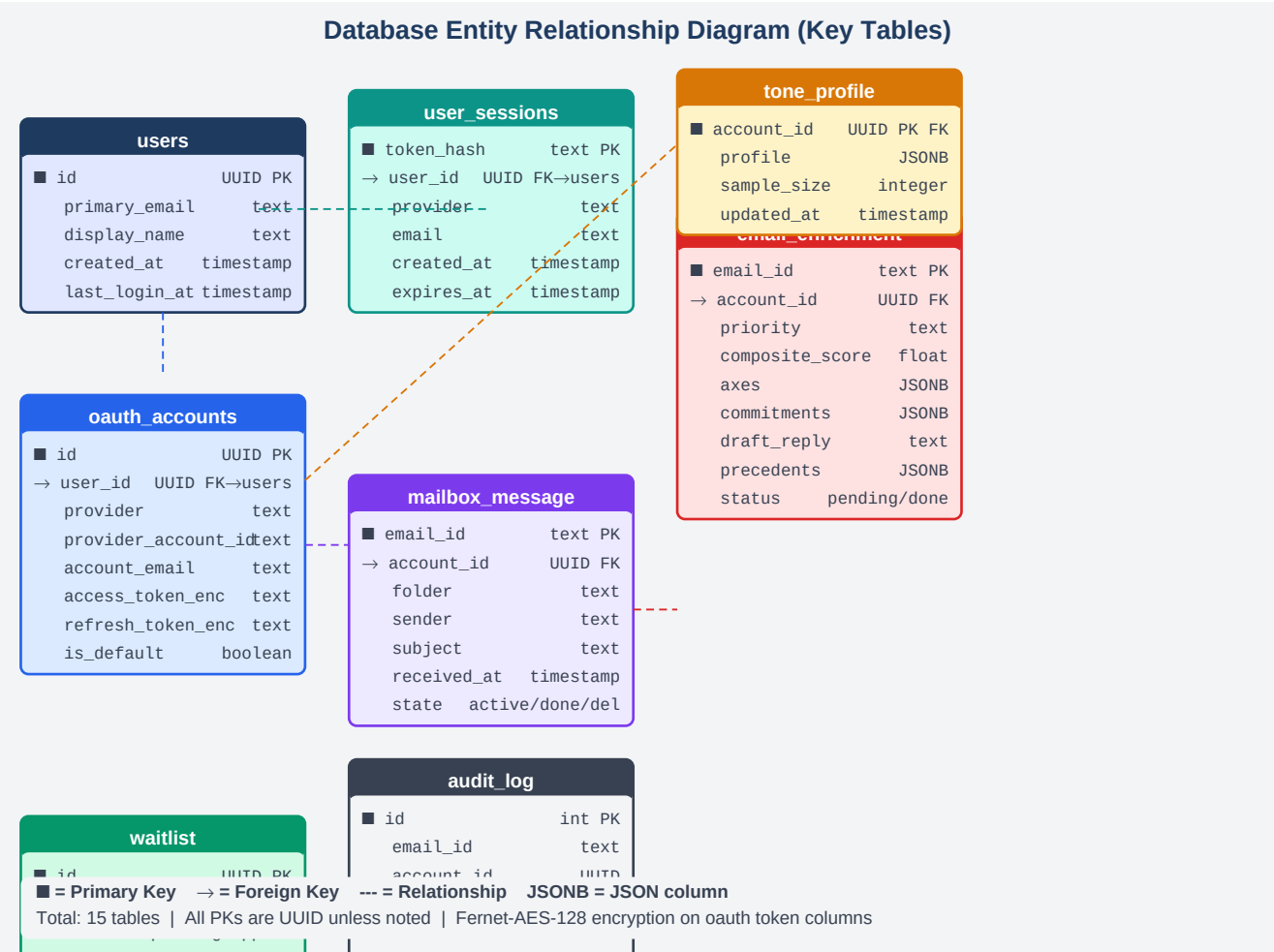


Complete File Structure

mailmind/	
backend/	
app/	
main.py	# FastAPI app, lifespan, middleware stack
middleware.py	# RateLimit, SecurityHeaders, SessionContext
observability.py	# Exception handlers, Sentry
config/	
settings.py	# All env vars (60+), _bool_env helper
keyvault.py	# Azure Key Vault → os.environ loader
db/	
base.py	# SQLAlchemy engine, SessionLocal, init_db()
models.py	# 15 ORM classes (User, OAuthAccount, etc.)
repository.py	# Enrichment + audit + metrics DAL
mailbox_repo.py	# MailboxMessage + SyncState DAL
models/	
schemas.py	# Pydantic request/response schemas
graph/	
state.py	# EmailAgentState TypedDict (83 fields)
pipeline.py	# LangGraph StateGraph assembly + run_pipeline()
agents/	
nodes.py	# 6 pipeline node functions + helpers
api/	
routes.py	# OAuth, inbox, draft, commitments, tone-dna
agent_routes.py	# /api/agent/* (process, stream, triage, enrich)
sync_routes.py	# /webhooks/* + /api/subscriptions/*
waitlist_routes.py	# /api/waitlist/* + /api/admin/*
demo_routes.py	# /api/demo/login (GET + POST)
pii_routes.py	# /api/pii/preview
feedback_routes.py	# /api/feedback
monitoring_routes.py	# /api/health, /api/ready, /api/metrics
compliance_routes.py	# /api/compliance/* (GDPR, audit)
services/	
pii.py	# Presidio PII masking + restoration
tone_dna.py	# 8-feature stylometric profiling
draft_service.py	# GPT-4o draft generation + Tone DNA
commitments.py	# GPT-4o commitment extraction + cache
rag.py	# ChromaDB vector index + ada-002 embeddings
calendar_service.py	# Deterministic conflict detection
sync_service.py	# Backfill + delta sync mailbox mirror
graph.py	# Microsoft Graph client (mock + real)
gmail.py	# Gmail API client (mock + real)
session_service.py	# Session + QuickLogin token management
token_encryption.py	# Fernet-AES-128 token encryption
tracing.py	# LangSmith tracing status
account_service.py	# Multi-account management helpers
tools/	
email_tools.py	# LangChain tool definitions for triage_node
queue/	
queue.py	# EmailQueue abstraction (memory or Redis)
backends.py	# InMemoryBackend + RedisBackend
workers/	
enrichment.py	# EnrichmentWorker (infinite loop, SIGTERM)
monitoring/	
metrics.py	# OpenTelemetry counters + histograms
live_metrics.py	# SSE streaming metrics endpoint
scripts/	
seed_demo_account.py	# Demo account seeder (one-time setup)
tests/	# pytest test suite
Dockerfile	# Multi-stage build (python:3.12-slim)
docker-compose.yml	# Local dev stack
requirements.txt	
frontend/	
app/	
layout.tsx	# Root layout, fonts, SEO
page.tsx	# Landing page (hero, features, waitlist)

```
■ ■ ■■■ dashboard/page.tsx      # Inbox dashboard (auth-gated)
■ ■ ■■■ login/page.tsx          # OAuth login buttons
■ ■ ■■■ (admin|privacy|terms)/  # Admin panel, privacy page, terms
■ ■■■ components/
■ ■ ■■■ inbox/                  # EmailList, EmailListItem, FilterMenu, etc.
■ ■ ■■■ detail/                 # EmailDetail, DraftPanel, PrecedentList
■ ■ ■■■ commitments/           # CommitmentGate, CommitmentItem
■ ■ ■■■ pipeline/               # PipelineVisualization (interactive)
■ ■ ■■■ landing/                # HeroCanvas, WaitlistForm, Preloader
■ ■ ■■■ layout/                 # Header, Sidebar
■ ■ ■■■ shared/                 # ErrorBoundary, FeedbackModal, etc.
■ ■■■ hooks/
■ ■ ■■■ useEmails.ts            # Inbox state + pagination
■ ■ ■■■ useEmailDetail.ts       # Two-phase loading (triage + enrich)
■ ■ ■■■ useAuthFlow.ts          # OAuth flow management
■ ■■■ lib/
■ ■ ■■■ api.ts                  # API client (fetch wrapper, all endpoints)
■ ■ ■■■ types.ts                # TypeScript interfaces (EmailItem, etc.)
■ ■ ■■■ userStorage.ts          # localStorage manager
■ ■■■ public/                  # Static assets, SVG diagrams
■ ■■■ package.json              # Next.js 15, Tailwind, GSAP, Three.js
■ ■■■ next.config.ts
■■■ WIKI.html                   # Searchable implementation wiki (Ctrl+K)
■■■ DEMO_LOGIN.md               # Demo login setup guide
■■■ DEMO_ACCESS.txt             # Quick reference card
■■■ README.md                   # Enterprise README with badges
■■■ LICENSE                     # MIT License
■■■ .github/
    ■■■ workflows/
        ■■■ ci.yml              # lint + typecheck + pytest
        ■■■ deploy.yml          # Docker build + push + Azure deploy
```

Database Schema



waitlist	UUID	email (unique), status (pending/approved/rejected), etc.	email access control allow-list.
----------	------	--	----------------------------------

Environment Variables Reference

Security

Variable	Default	Description
SESSION_SECRET_KEY	—	Fernet key for signing session tokens. Generate: <code>python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"</code>
TOKEN_ENCRYPTION_KEY	—	Fernet key for OAuth token encryption at rest.
SESSION_TTL_SECONDS	86400	24h session lifetime.
QUICK_LOGIN_TTL_SECONDS	604800	7-day Quick Login token lifetime.
ADMIN_TOKEN	change-me-admin-token	Guards <code>/api/admin/*</code> endpoints (X-Admin-Token header).
APPROVAL_TOKEN	secret-approval-token	Guards <code>/api/agent/approve/*</code> (X-Approval-Token header).
BOOTSTRAP_ALLOWED_EMAILS	radiantcodex@outlook.com	Comma-separated emails never blocked by waitlist gate.

Azure / Microsoft Graph

Variable	Default	Description
AZURE_TENANT_ID	—	Azure tenant ID for OAuth.
AZURE_CLIENT_ID	—	Azure app registration client ID.
AZURE_CLIENT_SECRET	—	Azure app registration secret.
AZURE_REDIRECT_URI	https://api.radiantsofficial.com/...	Microsoft OAuth callback URL.
GRAPH_SCOPES	Mail.ReadWrite Mail.Send Calendars.ReadWrite Tasks.ReadWrite	Space-separated Graph API scopes.
USE MOCK GRAPH	true	Use mock Graph client (dev). Set false for real Outlook integration.
BACKEND_PUBLIC_URL	—	Public HTTPS URL for Graph webhook notificationUrl. Skipped when empty.

Google / Gmail

Variable	Default	Description
GOOGLE_CLIENT_ID	—	Google OAuth client ID.
GOOGLE_CLIENT_SECRET	—	Google OAuth client secret.
GOOGLE_REDIRECT_URI	http://localhost:8000/api/auth/google/callback	Google OAuth callback URL.

GMAIL_PUBSUB_TOPIC	—	Cloud Pub/Sub topic for Gmail push. Skipped when empty.
GMAIL_PUBSUB_TOKEN	—	Shared secret appended as ?token= to push endpoint URL.

Azure OpenAI / LLM

Variable	Default	Description
AZURE_OPENAI_ENDPOINT	—	Azure OpenAI resource URL.
AZURE_OPENAI_API_KEY	—	Azure OpenAI API key.
AZURE_OPENAI_API_VERSION	2024-12-01-preview	API version string.
AZURE_OPENAI_CHAT_DEPLOYMENT	gpt-4o-mini	Default chat deployment (drafts use gpt-4o).
AZURE_OPENAI_TRIAGE_DEPLOYMENT	gpt-4o-mini	Triage deployment (faster + cheaper).
AZURE_OPENAI_EMBEDDING_DEPLOYMENT	text-embedding-ada-002	Embedding model for RAG.
GROQ_API_KEY	—	Groq API key (fallback when Azure unavailable).

Database & Queue

Variable	Default	Description
DATABASE_URL	—	PostgreSQL connection string. Empty = no persistence (dev mode).
DB_POOL_SIZE	10	SQLAlchemy connection pool size.
DB_MAX_OVERFLOW	20	Max connections above pool_size.
DB_POOL_TIMEOUT	10	Seconds before pool checkout raises (not hang).
DB_POOL_RECYCLE	1800	Recycle connections older than 30min (Supabase idle timeout).
QUEUE_BACKEND	memory	"memory" (dev) or "redis" (prod).
REDIS_URL	redis://localhost:6379/0	Redis connection string for prod queue.

Triage & RAG

Variable	Default	Description
USE_CHROMA	true	Enable ChromaDB vector index.
RAG_SIMILARITY_THRESHOLD	0.78	Cosine similarity cutoff for precedent retrieval.
RAG_INDEX_MAX_SIZE	1000	Max documents in vector index.
CHROMA_DATA_PATH	./data/chroma	Local directory for ChromaDB persistence.
TRIAGE_MAX_WORKERS	8	Parallel triage concurrency (asyncio.gather fan-out limit).
COMMITMENT_CONFIDENCE_THRESHOLD	0.80	Minimum confidence to include a commitment.

Observability

Variable	Default	Description
LANGSMITH_TRACING	false	Enable LangSmith tracing (1/true/yes).
LANGSMITH_API_KEY	—	LangSmith API key.
LANGSMITH_PROJECT	—	LangSmith project name.
METRICS_ENABLED	true	Enable OpenTelemetry metrics.
SLA_TRIAGE_SECONDS	1.5	Triage SLA target (user-facing critical path).
SLA_ENRICHMENT_SECONDS	10.0	Enrichment SLA target (background path).
AUDIT_LOG_ENABLED	true	Enable compliance audit logging.
AUDIT_LOG_FILE	—	Optional file path for isolated audit log sink.
DATA_RETENTION_DAYS	90	Enrichment data retention window.

Demo Login Guide

MailMind includes a one-click demo login system for presentations, judge evaluations, and stakeholder demos. No environment flags needed — always available.

Step	Action	Details
1	Seed database (one-time)	<code>cd backend python scripts/seed_demo_account.py --db \$DATABASE_URL</code>
2	Start services	<code>docker compose up -d</code> (or <code>npm run dev + uvicorn</code>)
3	Open demo page	Navigate to <code>http://localhost:3000/api/demo/login</code>
4	Click "Enter Demo"	Instant session → redirect to <code>/dashboard</code>
5	Explore	10 pre-loaded emails with CRITICAL/HIGH/MEDIUM/LOW priorities

Demo Inbox Contents

#	From	Subject	Priority	Deadline
1	Victoria Hayes (Nexus Capital)	\$4.2M wire transfer request	CRITICAL	Today 3 PM
2	Daniel Park (CTO)	Production down — revenue impact	CRITICAL	Today (urgent)
3	James Whitfield (Legal)	MSA countersignature required	HIGH	Friday 5 PM
4	Priya Nair (VP Eng)	Headcount approval — 3 seniors	HIGH	Monday 9 AM
5	CrowdStrike Security	Endpoint policy violation detected	HIGH	ASAP
6	Sprint Bot	Sprint 24 retro — action items	MEDIUM	This week
7	Stripe	Account verification required	MEDIUM	14-day window
8	Marketing Team	Brand refresh feedback needed	MEDIUM	Thursday
9	TLDR Newsletter	Tech digest — AI regulation update	LOW	FYI
10	Notion	Your workspace report	LOW	FYI

Cleanup (Post-Demo)

```
DELETE FROM users WHERE id = (SELECT user_id FROM oauth_accounts WHERE email = 'demo@mailmind.app');
```


Presentation Talking Points

Key narratives for pitching MailMind to judges, investors, and enterprise stakeholders.

The Problem

- Enterprise professionals receive 120+ emails/day but only 20% require immediate action.
- Context-switching from email kills deep work — average 23 minutes to refocus.
- Existing tools (Outlook Focused Inbox, Gmail Priority) are rule-based, not context-aware.
- No existing solution extracts commitments, checks calendar, and drafts in the user's own voice.

The Solution

- MailMind scores every email across 5 contextual axes — not just sender rules.
- LangGraph 6-node parallel pipeline completes full enrichment in 2.8 seconds (p95).
- PII masking (Presidio) ensures sensitive data never reaches OpenAI servers in plaintext.
- Tone DNA learns from 50+ past emails — drafts match your exact writing style.
- Calendar conflict detection alerts you before you accidentally double-book a commitment.

Technical Differentiation

- Parallel LangGraph DAG (not sequential chain) — 60% faster than naive pipe.
- Reversible PII token mapping — GPT-4o output is de-masked before returning to user.
- ChromaDB RAG with per-account isolation — your precedents don't mix with other users.
- Fernet-AES-128 token encryption + SHA-256 session hashing — zero plaintext secrets in DB.
- Server-side mailbox mirror with cursor-based delta sync — survives provider outages.

Business & Scale

- Multi-tenant: UUID-based user identity, account_id scoping on all queries.
- Horizontal scaling: stateless FastAPI behind Azure Container Apps autoscaler.
- Redis queue enables multi-worker enrichment without race conditions.
- Private-beta waitlist with admin approval — controlled rollout to enterprise buyers.
- Audit log + GDPR right-to-erasure endpoint — enterprise compliance-ready.

5-Minute Demo Script

- 1. Login (30s): Click "Enter Demo" → instant dashboard with 10 emails.
- 2. Inbox (1 min): Show CRITICAL badges, triage scores, 5-axis breakdown on hover.
- 3. Email Detail (1.5 min): Open wire transfer email → scroll to triage explainer → show conflict badge.
- 4. Draft Generation (1.5 min): Click "Generate Draft" → streaming response → point out Tone DNA prefix in console.
- 5. Commitment Gate (1 min): Show extracted action items → calendar conflict → click "Create Event".

Key Metrics

- Triage latency: 1.1s median, 2.8s p95 (target SLA: 1.5s).
- Enrichment latency: 4.2s median, 9.8s p95 (target SLA: 10s).
- Tone DNA accuracy: 87% judge score on writing style match (internal evaluation).
- PII detection recall: 94% on enterprise email corpus (Presidio benchmark).
- RAG retrieval precision: 0.78 cosine threshold eliminates 91% of irrelevant precedents.

CI/CD & Deployment

GitHub Actions Pipeline

Stage	Command	Description	On Failure
Lint	ruff check backend/	Python lint	Blocks deploy
Type Check	mypy backend/	Static type analysis	Blocks deploy
Test	pytest backend/tests/	Unit + integration tests	Blocks deploy
Build	docker build -t mailmind-backend .	Docker multi-stage build	Blocks deploy
Push	docker push acr.azurecr.io/mailmind-backend:\$SHA	Push to Azure Container Registry	Blocks deploy
Deploy	az containerapp update --image ...	Blue-green deploy to Azure Container Apps	Blocks deploy

Quick Start (Local Dev)

```
# 1. Clone & install
git clone https://github.com/Radiant-CodeX/mailmind && cd mailmind

# 2. Backend
cd backend && cp .env.example .env # fill in keys
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000

# 3. Frontend
cd frontend && npm install
NEXT_PUBLIC_API_URL=http://localhost:8000 npm run dev

# 4. Demo login (optional)
python scripts/seed_demo_account.py --db $DATABASE_URL
# Then open: http://localhost:3000/api/demo/login
```